

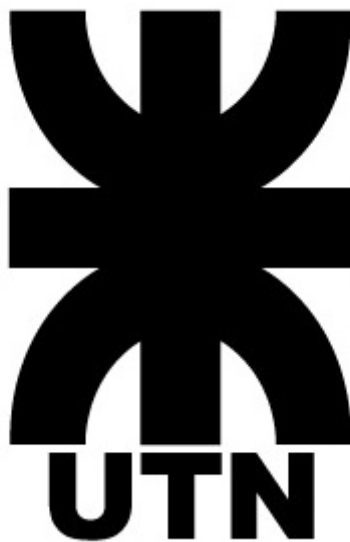
UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

Trabajo Práctico Integrador

Gestión de Datos de Países en Python: Filtros, Ordenamientos y Estadísticas

Materia	Programación 1
Integrantes	Francisco Nahuel Ortiz Fernando Angel Torres Fernandez
Comisión / Grupo	Grupo N° 95
Fecha de entrega	Junio 2026



Índice

1. Marco Teórico	3
1.1 Estructuras de Control	3
1.2 Manejo de Errores	3
1.3 Manejo de Archivos CSV	3
1.4 Listas	4
1.5 Diccionarios	4
1.6 Funciones	4
1.7 Validación de Datos	4
1.8 Ordenamientos	5
1.9 Evitar Duplicados	5
1.10 Interacción con el usuario	5
2. Decisiones Técnicas y Arquitectura	6
2.1 Diagrama de Flujo	6
2.2 Capturas de Pantalla	6
3. Dificultades y Conclusiones	7
4. Participación de los Integrantes	8
5. Bibliografía / Webgrafía	9
6. Links del Proyecto	9

1. Marco Teórico

El programa presentado por los integrantes Ortiz Francisco Nahuel y Torres Fernandez Fernando Angel fue desarrollado en Python con el objetivo de gestionar información sobre países mediante el uso de archivos CSV. Se aplicaron diferentes funcionalidades: el agregado, búsqueda, actualización, filtrado, ordenamiento y visualización de estadísticas, haciendo uso de los conocimientos provistos por la materia Programación 1.

1.1 Estructuras de Control

En el programa se utilizan estructuras de control para dirigir la ejecución del flujo:

- Condicionales (if, elif, else):
 - Verificar campos vacíos.
 - Controlar rangos de valores.
 - Determinar opciones seleccionadas por el usuario.
- Bucles (while, for):
 - While: validar entradas hasta que sean correctas.
 - For: recorrer los datos del archivo CSV.

1.2 Manejo de Errores

Se hace uso de bloques try–except para controlar errores durante la ejecución, capturando los siguientes tipos de excepciones:

- ValueError: cuando el usuario ingresa datos de tipo incorrecto.
- FileNotFoundError: cuando el archivo CSV no existe en la ruta indicada.
- IOError: cuando ocurren errores al intentar escribir en el archivo.
- Raise: Lanzar excepciones con mensajes personalizados.

Esto permite evitar que el programa se detenga de manera abrupta, mostrando mensajes claros y descriptivos al usuario ante cada situación de error.

1.3 Manejo de Archivos CSV

El programa utiliza archivos CSV para almacenar y persistir los datos de manera estructurada. Se utiliza el módulo csv, incluido en la biblioteca estándar de Python, sin necesidad de instalar dependencias externas. Las herramientas utilizadas son:

- csv.reader: lectura de filas como listas.
- csv.writer: escritura de datos en formato lista.
- csv.DictWriter: escritura de datos usando diccionarios, respetando el orden de los campos definidos en fieldnames.

Cada fila del archivo representa un país con el siguiente formato:

```
nombre,poblacion,superficie,continente  
Argentina,45376763,2780400,América
```

1.4 Listas

Los datos leídos desde el archivo CSV se almacenan en listas de Python para ser procesados en memoria. Esto permite recorrerlos, filtrarlos, ordenarlos y mostrarlos sin necesidad de acceder al archivo en cada operación. Python permite almacenar en listas tanto tipos simples (strings, enteros) como estructuras más complejas como diccionarios.

1.5 Diccionarios

Cada país se representa internamente como un diccionario de Python, donde las claves corresponden a los campos del dataset: nombre, poblacion, superficie y continente. Esto facilita el acceso a cada dato de forma clara y legible, por ejemplo: `pais['nombre']` o `pais['poblacion']`.

1.6 Funciones

El programa está organizado mediante funciones, siguiendo el principio de una función = una responsabilidad. Esto mejora la legibilidad, el mantenimiento y la reutilización del código. Algunas de las funciones principales son:

- `agregar_pais()`: solicita y valida los datos de un nuevo país y lo agrega al CSV.
- `actualizar_datos()`: busca un país y actualiza su población y superficie.
- `buscar_pais()`: permite buscar por nombre con coincidencia parcial o exacta.
- `filtrar_paises()`: filtra por continente, rango de población o superficie.
- `ordenar()`: ordena el dataset por nombre, población o superficie.
- `mostrar_estadisticas()`: calcula y muestra indicadores clave del dataset.
- `pais_existe()`: función auxiliar que verifica si un país ya está cargado.

1.7 Validación de Datos

Se aplican validaciones para asegurar la integridad de la información ingresada, evitando:

- Campos vacíos en cualquier campo del formulario.
- Valores negativos o cero en población y superficie.
- Opciones inválidas en el menú principal.
- Ingreso de números donde se esperan nombres o continentes.
- Carga de países duplicados mediante la función `pais_existe()`.

1.8 Ordenamientos

Para ordenar los datos se utiliza el método `.sort()` combinado con funciones lambda como clave de comparación. Las funciones lambda permiten definir el criterio de ordenamiento de manera concisa. El parámetro `reverse=True` permite invertir el orden para resultados descendentes. El dataset puede ordenarse por:

- Nombre (alfabético).

Trabajo Practico Integrador

- Población (numérico).
- Superficie (numérico).

1.9 Evitar Duplicados

Se desarrolló la función auxiliar `pais_existe()` que recorre el CSV antes de agregar un nuevo país, comparando los nombres sin distinguir mayúsculas mediante `.lower()`. Si el país ya existe, se informa al usuario y se interrumpe la operación, evitando inconsistencias en la base de datos.

1.10 Interacción con el usuario

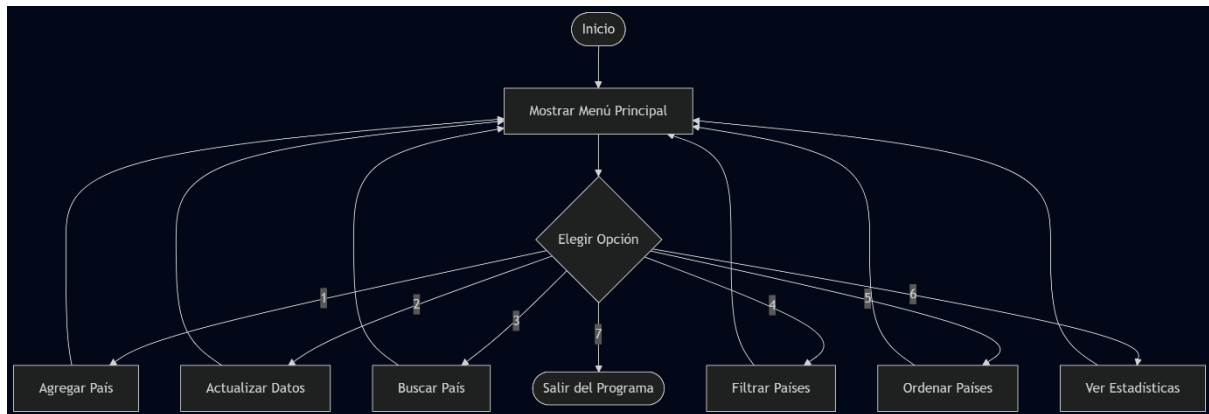
El programa es completamente interactivo mediante consola, haciendo uso de:

- `input()`: para el ingreso de datos por teclado.
- `print()`: para mostrar información, resultados y mensajes al usuario.
- `match/case`: para implementar el menú principal de manera estructurada (disponible desde Python 3.10).

2. Decisiones Técnicas y Arquitectura

2.1 Diagrama de Flujo

El siguiente diagrama representa el flujo general de operaciones del programa:



2.2 Capturas de Pantalla

A continuación, se muestran capturas de pantalla del programa en ejecución con los distintos flujos:

Menú principal:

```
-----
1-Agregar país.
2-Actualizar datos(Poblacion y superficie).
3-Buscar país.
4-Filtrar países.
5-Ordenar países.
6-Ver estadísticas.
7-Salir.

Ingresa opcion: 
```

Agregar país:

Trabajo Practico Integrador

```
Ingresa opcion: 1
```

```
--- Agregar nuevo país ---
```

```
Nombre del país: Bolivia
```

```
Población: 1245121
```

```
Superficie en km²: 1243453
```

```
Continente: America
```

```
País 'Bolivia' agregado correctamente.
```

Buscar país (coincidencia parcial):

```
Ingresa opcion: 3
```

```
--- Buscar país ---
```

```
Ingrese nombre a buscar: Bol
```

```
Resultados:
```

```
Nombre: Bolivia | Población: 1245121 personas | Superficie: 1243453km2 | Continente: America
```

Filtrar por continente:

```
Ingresa opcion: 4
```

```
--- Filtrar países ---
```

```
1 - Por continente
```

```
2 - Por rango de población
```

```
3 - Por rango de superficie
```

```
Seleccione una opción: 1
```

```
Ingrese continente: America
```

```
Se encontraron 5 país(es):
```

```
Nombre: Argentina | Población: 45376763 | Superficie: 2780400 | Continente: America
```

```
Nombre: Brasil | Población: 213993437 | Superficie: 8515767 | Continente: America
```

```
Nombre: Colombia | Población: 124242124 | Superficie: 12412343 | Continente: America
```

```
Nombre: Nicaragua | Población: 234234 | Superficie: 4234234 | Continente: America
```

```
Nombre: Bolivia | Población: 1245121 | Superficie: 1243453 | Continente: America
```

Ordenar por superficie descendente:

Trabajo Practico Integrador

```
Ingresa opcion: 5

--- Ordenar países ---
1 - Nombre
2 - Población
3 - Superficie
Seleccione criterio: 3
¿Ascendente (A) o Descendente (D)?: d

Países ordenados:

Nombre: Colombia | Población: 124242124 Personas | Superficie: 12412343 km2 | Continente: America
Nombre: Brasil | Población: 213993437 Personas | Superficie: 8515767 km2 | Continente: America
Nombre: Nicaragua | Población: 234234 Personas | Superficie: 4234234 km2 | Continente: America
Nombre: Argentina | Población: 45376763 Personas | Superficie: 2780400 km2 | Continente: America
Nombre: Bolivia | Población: 1245121 Personas | Superficie: 1243453 km2 | Continente: America
Nombre: Nigeria | Población: 223804632 Personas | Superficie: 923768 km2 | Continente: Africa
Nombre: Japon | Población: 125800000 Personas | Superficie: 377975 km2 | Continente: Asia
Nombre: Alemania | Población: 83149300 Personas | Superficie: 357022 km2 | Continente: Europa
```

Ver estadísticas:

```
Ingresa opcion: 6

--- Estadísticas ---

País con mayor población: Nigeria (223,804,632 habitantes)
País con menor población: Nicaragua (234,234 habitantes)
Promedio de población: 102,230,701 habitantes
Promedio de superficie: 3,855,620 km²

Cantidad de países por continente:
  America: 5 país(es)
  Europa: 1 país(es)
  Asia: 1 país(es)
  Africa: 1 país(es)
```


3. Dificultades y Conclusiones

Dificultades encontradas

Durante el desarrollo del proyecto surgieron distintos obstáculos técnicos que requirieron investigación y prueba para ser resueltos:

- Manejo del encabezado del CSV: al principio las funciones de búsqueda y filtrado incluían la fila de encabezado (nombre, poblacion, superficie, continente) como si fuera un país. Esto se resolvió incorporando `next(reader)` al inicio de cada lectura para saltar la primera fila.
- Indentación y lógica del `while True`: en la función `actualizar_datos()` el bucle de validación de los nuevos datos estaba incorrectamente fuera del bloque `if`, lo que hacía que se le pidieran datos nuevos a todos los países del CSV y no solo al encontrado. La solución fue revisar cuidadosamente la indentación del código.
- Manejo del `raise ValueError` con mensaje personalizado: al usar `raise ValueError('mensaje')` dentro de un `try/except ValueError`, el mensaje personalizado se perdía porque el `except` lo capturaba con su propio `print`. Se resolvió separando la captura del `int()` de la validación de lógica.
- Reescritura del archivo CSV: al actualizar datos fue necesario leer todo el archivo en memoria, modificar la fila correspondiente y reescribir el archivo completo, ya que CSV no permite edición directa por posición.

Conclusiones

El desarrollo de este Trabajo Práctico Integrador nos permitió consolidar los conceptos fundamentales de Programación 1 aplicados a un caso concreto. A lo largo del proyecto pudimos:

- Comprender la importancia de modularizar el código mediante funciones con responsabilidad única.
- Aplicar estructuras de datos como listas y diccionarios para gestionar información de manera eficiente.
- Entender el manejo de archivos CSV como mecanismo de persistencia de datos.
- Incorporar buenas prácticas de validación de entradas y manejo de errores para construir programas más robustos.
- Trabajar de manera colaborativa, dividiendo responsabilidades y revisando el código entre los dos integrantes.

El resultado final es un programa funcional, comentado y organizado que cumple con todas las funcionalidades solicitadas en la consigna.

4. Participación de los Integrantes

Integrante	Participación
Francisco Nahuel Ortiz	• Funciones agregar_pais() y pais_existe() • Funciones actualizar_datos() y buscar_pais() • Funciones filtrar_paises() y ordenar()
Fernando Angel Torres Fernandez	• Creación del repositorio del proyecto. • Funciones agregar_pais() y mostrar_estadisticas() • Función opcion() y estructura principal del programa. • Dataset CSV y archivo README
Desarrollado en conjunto	Estructura general del programa, validaciones, manejo de errores y documentación académica.

5. Bibliografía / Webgrafía

- Documentación oficial de Python 3 — Módulo csv:
<https://docs.python.org/3/library/csv.html>
- Documentación oficial de Python 3 — Built-in Exceptions:
<https://docs.python.org/3/library/exceptions.html>
- Documentación oficial de Python 3 — match/case (Structural Pattern Matching):
<https://docs.python.org/3/whatsnew/3.10.html#pep-634-structural-pattern-matching>
- Documentación oficial de Python 3 — Funciones integradas (sorted, max, min, sum):
<https://docs.python.org/3/library/functions.html>
- Real Python — Reading and Writing CSV Files in Python:
<https://realpython.com/python-csv/>

6. Links del Proyecto

**Repositorio
GitHub**

https://github.com/Anyeloou/TP_Integrador-Programacion-I

**Video
demostrativo**

<https://www.youtube.com/watch?v=yo5gB9nnwUA>